

Project Title: Digital Complaint Management & Grievance Portal

Objective

Develop a full-stack web application where users can register complaints, track their status, and provide feedback, with admin assignment and analytics (optional), using Angular, Node.js (TypeScript), and MySQL.

Functional Requirements

1. User Roles

- **User (Resident/Student/Employee):** Can register complaints, track status, and provide feedback.
- **Staff/Technician:** Can view assigned complaints, update status, and add resolution notes.
- **Admin (Optional):** Can assign complaints, monitor all complaints, manage users, and view analytics.

Registration Options:

Users choose their role at signup: **User / Staff / Admin (optional)**.

2. Complaint Registration & Management

Users can submit complaints with:

- Title, description, category (plumbing, electrical, facility, etc.)

- Optional attachments (store as **file path/URL**)
- Admin can assign complaints to staff/technicians.

UI Suggestions:

- **Complaint Registration Page:** Form to submit a complaint.
 - **User Dashboard:** Track complaints and view status.
 - **Admin Dashboard (Optional):** Assign complaints and view complaint list.
-

3. Complaint Status Tracking

- Complaint lifecycle: **Open** → **Assigned** → **In-progress** → **Resolved**
- Users get updates when status changes.
- Staff/Technicians can update status and add resolution notes.

UI Suggestions:

- **Status Tracking Page:** Shows status progression.
 - **Notes Section:** Staff can add updates or resolution details.
-

4. Frontend Requirements (Angular 16)

Use Angular Material for all UI components.
Maintain a clean folder/component hierarchy.

Suggested Components

- **RegistrationComponent** → Register new users with role selection.

- **ComplaintListComponent** → View complaints (User).
- **ComplaintDetailsComponent** → Submit/view individual complaint details.
- **StaffDashboardComponent** → Manage assigned complaints (Staff).
- **AdminDashboardComponent (Optional)** → Assign complaints & view analytics.

Suggested Routes

Path	Component	Description
/register	RegistrationComponent	User registration & role selection
/complaints	ComplaintListComponent	View all user complaints
/complaints/new	ComplaintDetailsComponent	Submit a new complaint
/staff/dashboard	StaffDashboardComponent	Manage assigned complaints
/admin/dashboard (optional)	AdminDashboardComponent	Assign complaints & analytics

5. Backend Requirements (Node.js + TypeScript + Express)

- Use MySQL for persistent storage.

- Create REST APIs for complaints, users, and staff assignment.
 - Include validation and error handling.
 - Admin-related APIs are **optional** depending on implementation.
-

6. Database Structure (MySQL)

Users Table

- id
- name
- role (User / Staff / Admin - **Admin optional**)
- contact_info
- created_at

Complaints Table

- id
- user_id (relates to Users)
- staff_id (**nullable**, optional assignment)
- title
- description
- category
- status (Open / Assigned / In-progress / Resolved)
- attachments (**optional**, file path/URL)
- created_at

Maximum 2 tables; simple relation between Users ↔ Complaints.

7. Validation Outline

- Complaint title, description, and category must not be empty.
 - Status must follow defined flow: **Open** → **Assigned** → **In-progress** → **Resolved**.
 - Role at registration must be valid: **User / Staff / Admin (optional)**.
 - Include all **necessary validation** to maintain workflow and data integrity.
-

8. Additional Requirements

- Proper HTTP status codes.
 - UI must show meaningful success or error messages.
 - Real-time or periodic updates for complaint status (**polling optional**).
 - Admin analytics dashboard is **optional**.
-

9. Exception Handling

- Frontend and backend should gracefully handle all errors.
 - UI must show user-friendly error messages.
 - Backend should log errors for debugging and maintenance.
-

10. Role-Based Access (Using Angular Guards)

Use Angular route guards to control access:

- **User:** Submit complaints, view status, give feedback.
- **Staff/Technician:** View assigned complaints, update status.
- **Admin (Optional):** Assign complaints, view analytics & overall system data.

Unauthorized users are redirected to **login** or a **Not Authorized** page.

Guards ensure proper role-based component access and secure workflow.